

Algorytmy macierzowe

Mnożenie macierzy gęstych

Prof. Dr hab. Maciej Paszyński

Wydział Informatyki AGH

Jeśli używasz fragmentów tego wykładu, zacytuj
źródło

Macierze gęste

- Przechowujemy wszystkie wartości macierzy
- Mnożenie macierzy: algorytm „klasyczny”, algorytm Strassena, algorytm Coppersmith-Winograda (galaktyczny)
- Złożoność klasycznego algorytmu mnożenia macierzy
 $C(m \times n) = A(m \times k) * B(k \times n)$
to $O(m*n*k)$
- Koszt algorytmu to czas wykonania $2*m*n*k$ operacji zmiennie-przecinkowych PLUS czas przesyłania danych z pamięci RAM do rejestrów procesora
- Efektywność implementacji wymaga bardzo uważnego przemyślenia organizacji danych w pamięci komputera

Mnożenie macierzy gęstych

- Źródło

<https://www.cs.utexas.edu/users/flame/laff/pfhp>

Prof. Robert van de Geijn

https://en.wikipedia.org/wiki/Robert_van_de_Geijn



----- Wiadomość oryginalna -----

Temat: Re: Greetings

Data: 2019-04-03 20:09

Od: Robert van de Geijn <rvdg@cs.utexas.edu>

Do: paszynsk@agh.edu.pl

> On Apr 3, 2019, at 10:47 AM, Maciej Paszynski <paszynsk@agh.edu.pl> wrote:

>

> Dear Robert

> Looks like from the next year 2020 I will have two lectures,
Matrix algorithm and Matrix computations, for CS students.

> I am thinking of using your LAFF material for these courses
(plus some sparse solvers materials)

> and I would like to ask you kindly for permission.

We will be delighted to have you use it.

(...)

Robert

Mnożenie macierzy gęstych

flop = liczba operacji zmiennie-przecinkowych
(cecha algorytmu)

FLOPS = liczba operacji zmiennie-przecinkowych na sekundę
(cecha wykonania implementacji algorytmu na komputerze)

GFLOPS = liczba miliardów FLOPSów = $10^9 * \text{FLOPS}$

T[s] = zmierzony czas działania programu w sekundach

Jak przeliczyć T[s] (czas [sekundy] $\rightarrow$ GFLOPS

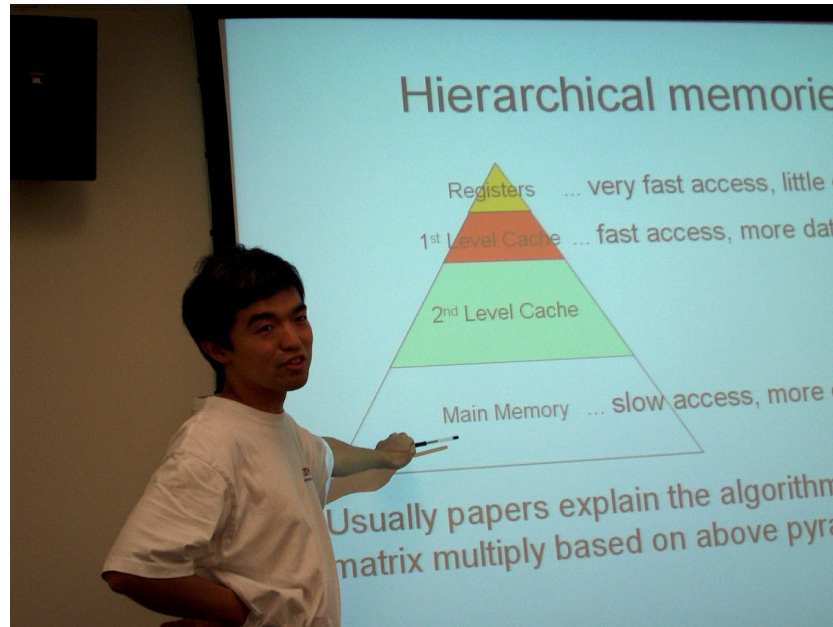
Mnożenie macierzy to $2 * m * n * p$ flopsów

(dodawanie i mnożenie zagnieżdżone w 3 pętlach)

$2 * m * n * p / T[s] = \text{FLOPS} = 10^{-9} * \text{GFLOPS}$

$\text{GFLOPS} = 10^9 * 2 * m * n * p / T[s]$

Matrix*Matrix multiplication



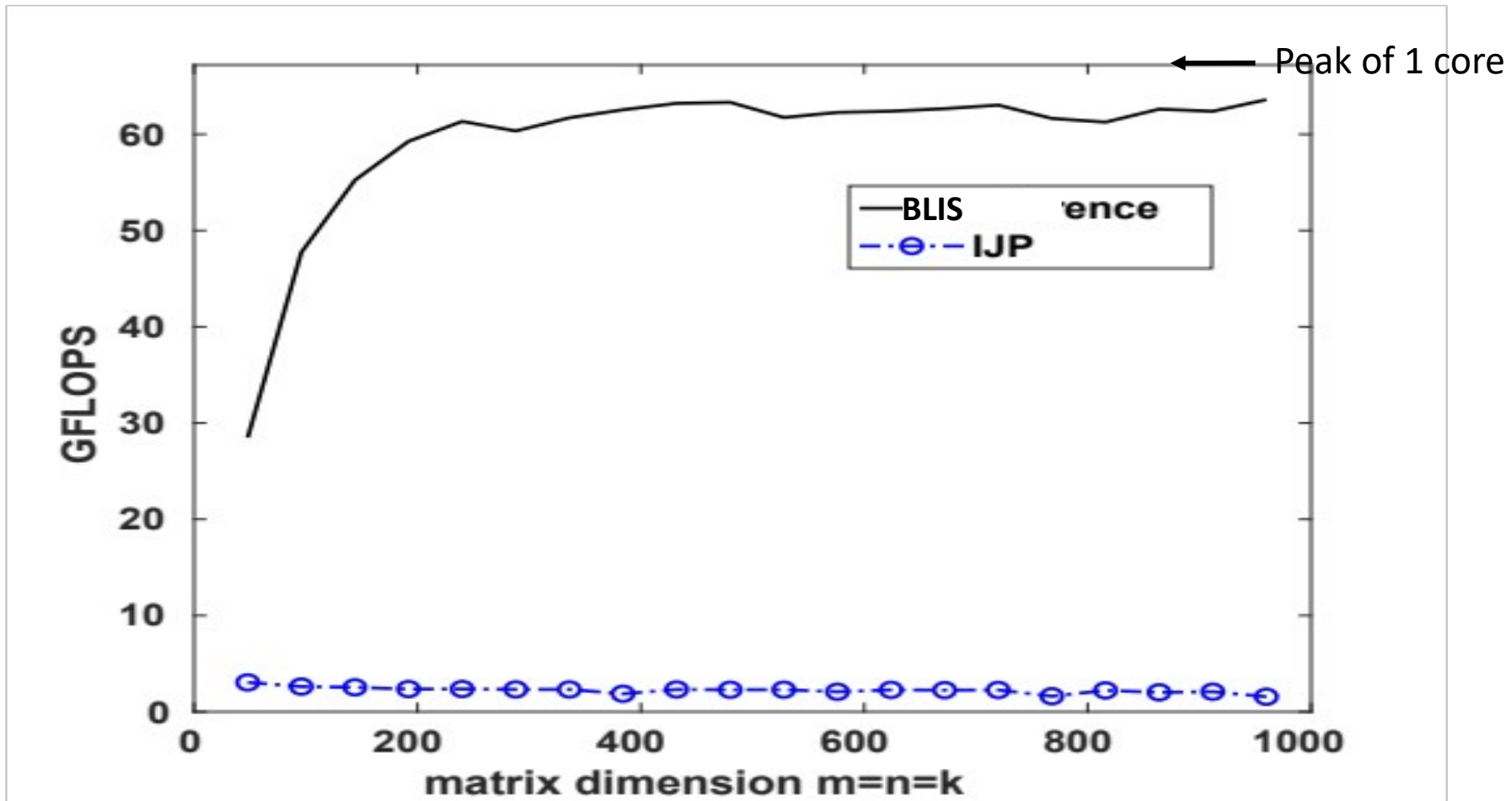
Japoński inżynier pracujący w urzędzie patentowym.

Opracował bibliotekę GotoBLAS do szybkiego mnożenia macierzy.

W 2007 roku dostał ofertę pracy od prof. Roberta van de Geijna na Uniwersytecie Teksanskim w Austin. Od 2010 roku pracuje w Microsoft.

Goto, Kazushige; van de Geijn, Robert A. (2008), "Anatomy of High-Performance Matrix Multiplication", ACM Transactions on Mathematical Software, 34 (3)Article 12 [jedno z najlepszych czasopism na świecie w dyscyplinie]

Use a high-performance library!



The Basic Linear Algebra Subprograms (BLAS)

- Level-1 BLAS (vector-vector operations)

C.L. Lawson, R.J. Hanson, D.R. Kincaid, and F.T. Krogh. Basic linear algebra subprograms for Fortran usage. ACM TOMS. 1979.

- Level-2 BLAS (matrix-vector operations)

J.J. Dongarra, J. Du Croz, S. Hammarling, R.J. Hanson. An extended set of FORTRAN basic linear algebra subprograms. ACM TOMS. 1988

- Level-3 BLAS (matrix-matrix operations)

J. J. Dongarra, J. Du Croz, S. Hammarling, and I. Duff. A set of level 3 basic linear algebra subprograms. ACM TOMS, 1990

The BLAS are the building blocks for scientific computing (and machine learning)

Open Source Efforts

Use optimized small GEMM to build high-performing level-3 BLAS.

- “Poor man’s BLAS”

B. Kagstrom, P. Ling, and C. Van Loan. GEMM-based level 3 BLAS: High performance model implementations and performance evaluation benchmark. ACM TOMS. 1998.

Late 1990s:

- Portable High Performance ANSI C (PHiPAC)

J. Bilmes, K. Asanovic, C. Chin, and J. Demmel. Optimizing matrix multiply using PHiPAC: A portable, High-Performance, ANSI C coding methodology. ICS'97, 1997.

- Automatically Tuned Linear Algebra Subprograms (ATLAS)

R. C. Whaley and J.J. Dongarra. Automatically tuned linear algebra software. SC'98. 1998.

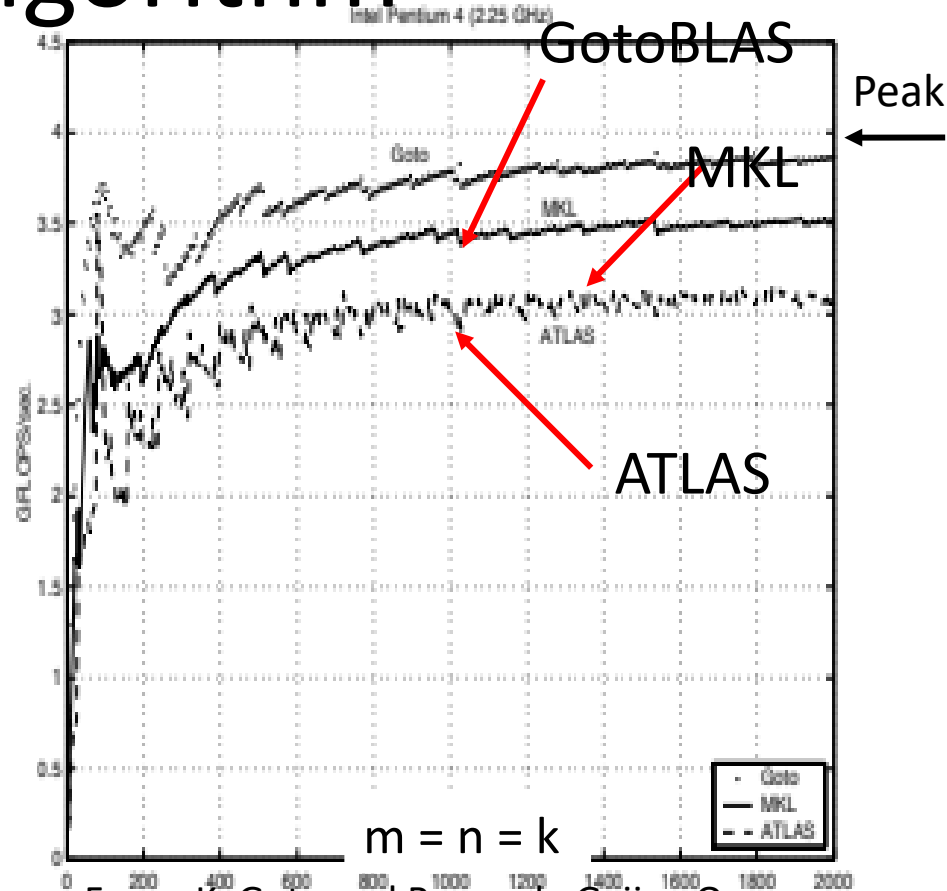
Goto's Algorithm

- 2002: Kazushige Goto reinvents how GEMM is implemented

K. Goto and R. van de Geijn. Anatomy of high-performance matrix multiplication. ACM TOMS. 2008.

K. Goto and R. van de Geijn. High-performance implementation of the level-3 BLAS. ACM TOMS. 2008.

- Distributed as GotoBLAS
- Forked as OpenBLAS



From: K. Goto and R. van de Geijn. On Reducing TLB Misses in Matrix Multiplication. FLAME Working Note #9. Nov. 2002.

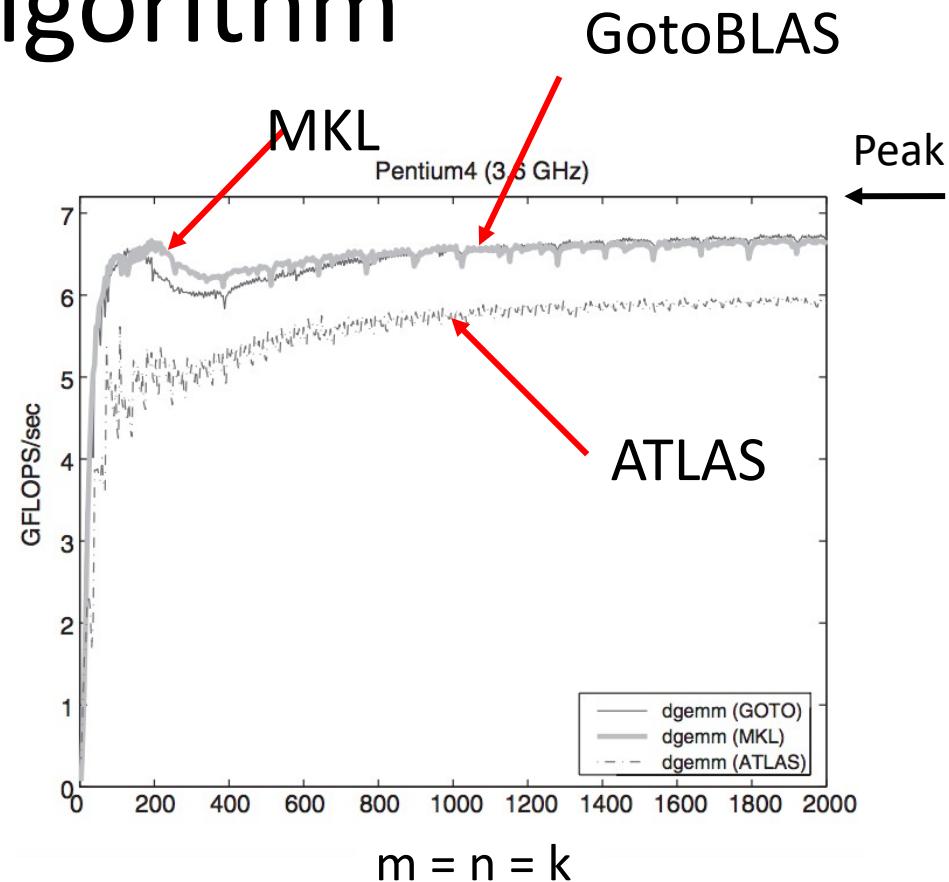
Goto's Algorithm

- 2002: Kazushige Goto reinvents how GEMM is implemented

K. Goto and R. van de Geijn. Anatomy of high-performance matrix multiplication. ACM TOMS. 2008.

K. Goto and R. van de Geijn. High-performance implementation of the level-3 BLAS. ACM TOMS. 2008.

- Distributed as GotoBLAS
- Forked as OpenBLAS



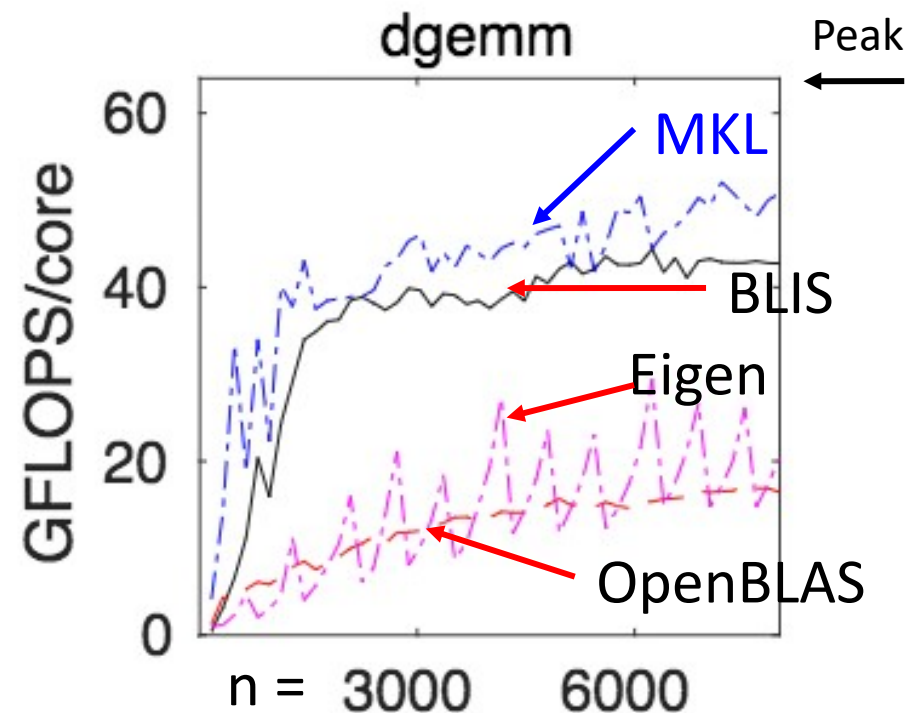
From: K. Goto and R. van de Geijn.
Anatomy of high-performance matrix
multiplication. ACM TOMS. 2008.

BLAS-like Library Instantiation Software (BLIS)

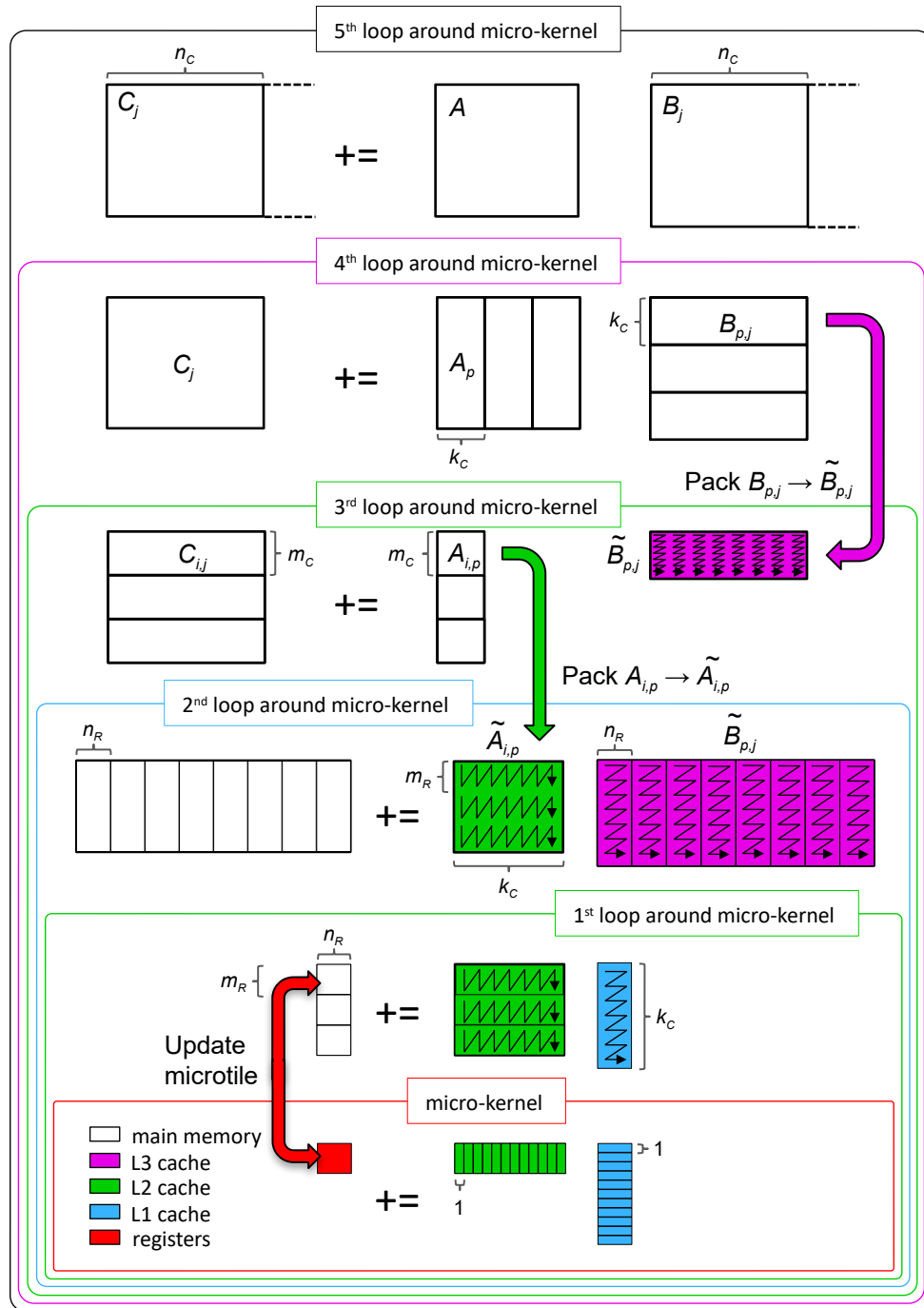
- Framework for implementing BLAS-like functionality

F. G. Van Zee, R. A. van de Geijn.
BLIS: A framework for rapidly instantiating BLAS functionality.
ACM TOMS, 2015.

- Refactoring of Goto's algorithm
- <https://github.com/flame/blis>



Intel SkylakeX (56 cores)
From: <https://github.com/flame/blis/>



Strassen's Algorithm

V. Strassen, "Gaussian elimination is not optimal," Numer. Math. 1969

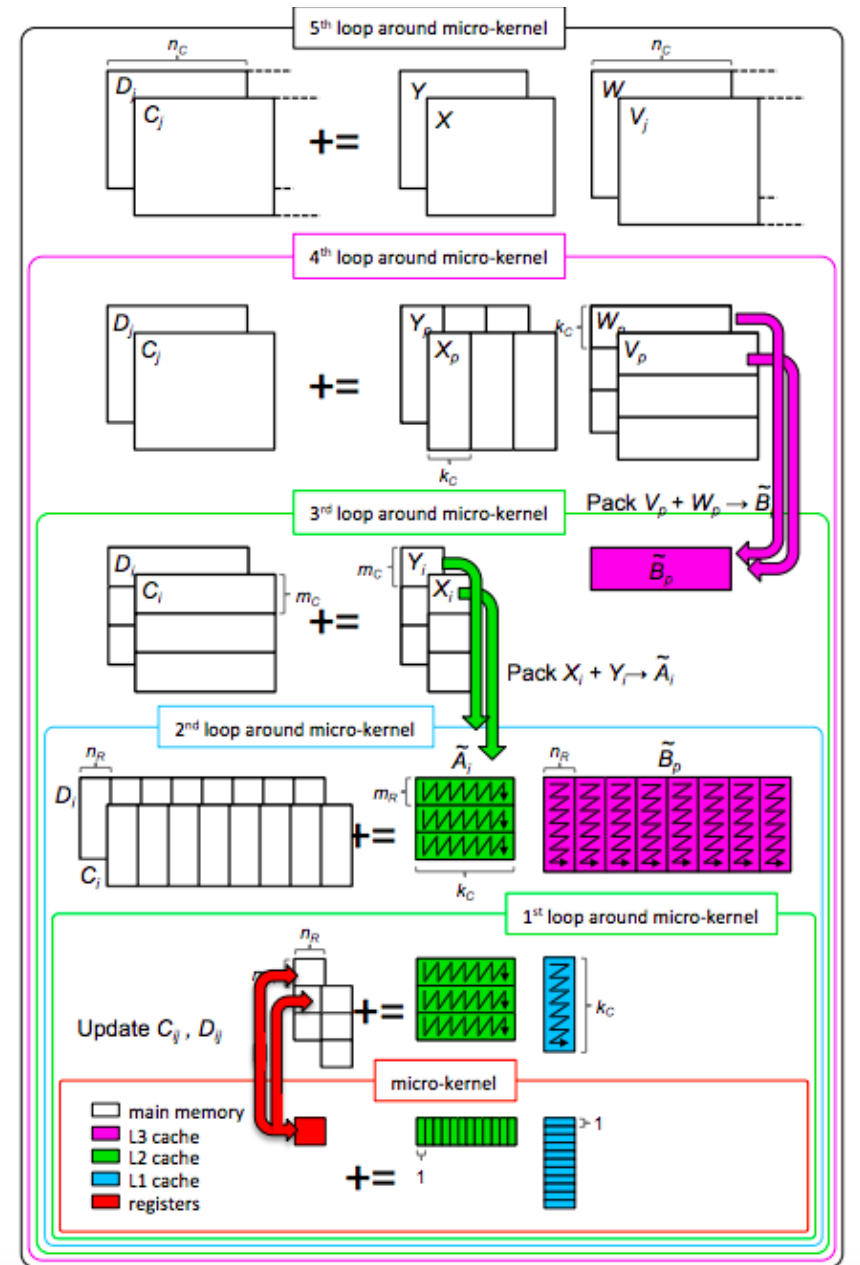
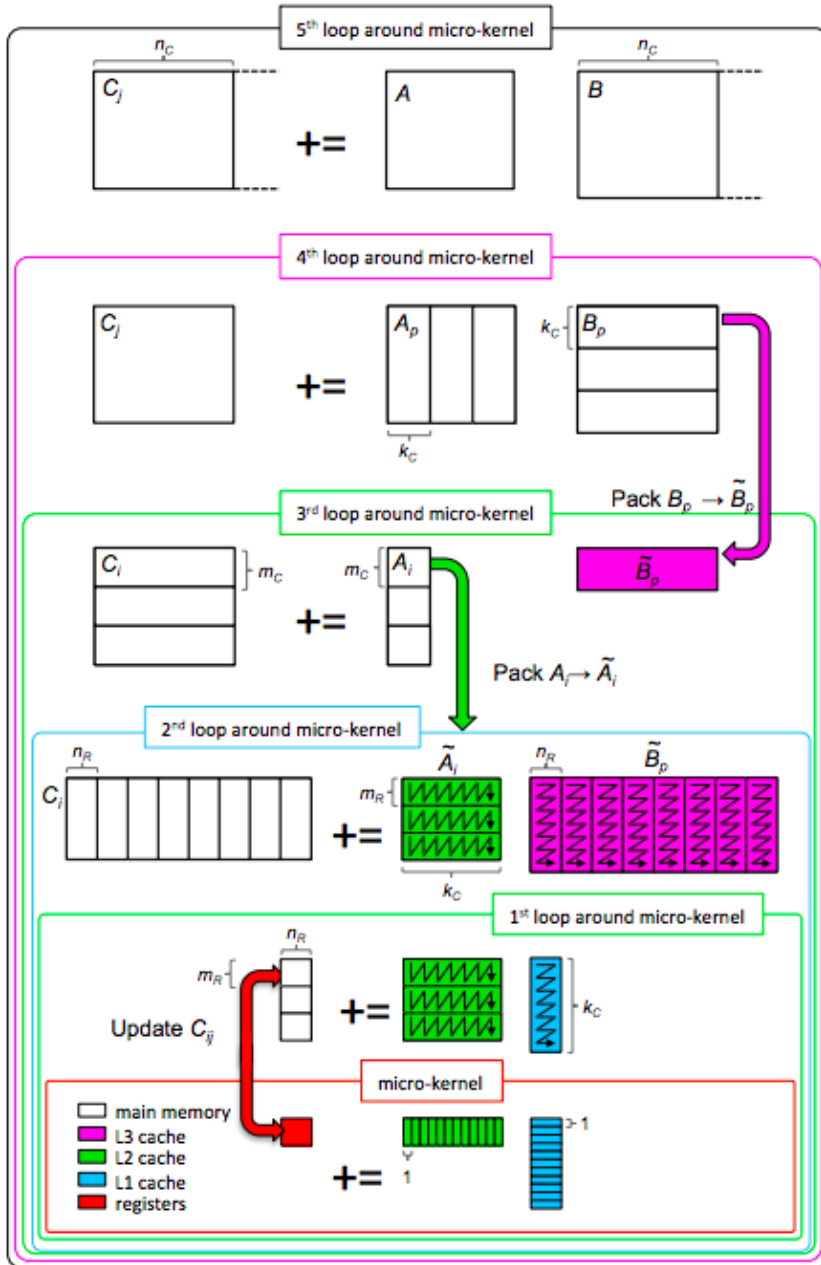
$$C = AB$$

$$C = \begin{pmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{pmatrix}, A = \begin{pmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{pmatrix}, B = \begin{pmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{pmatrix}$$

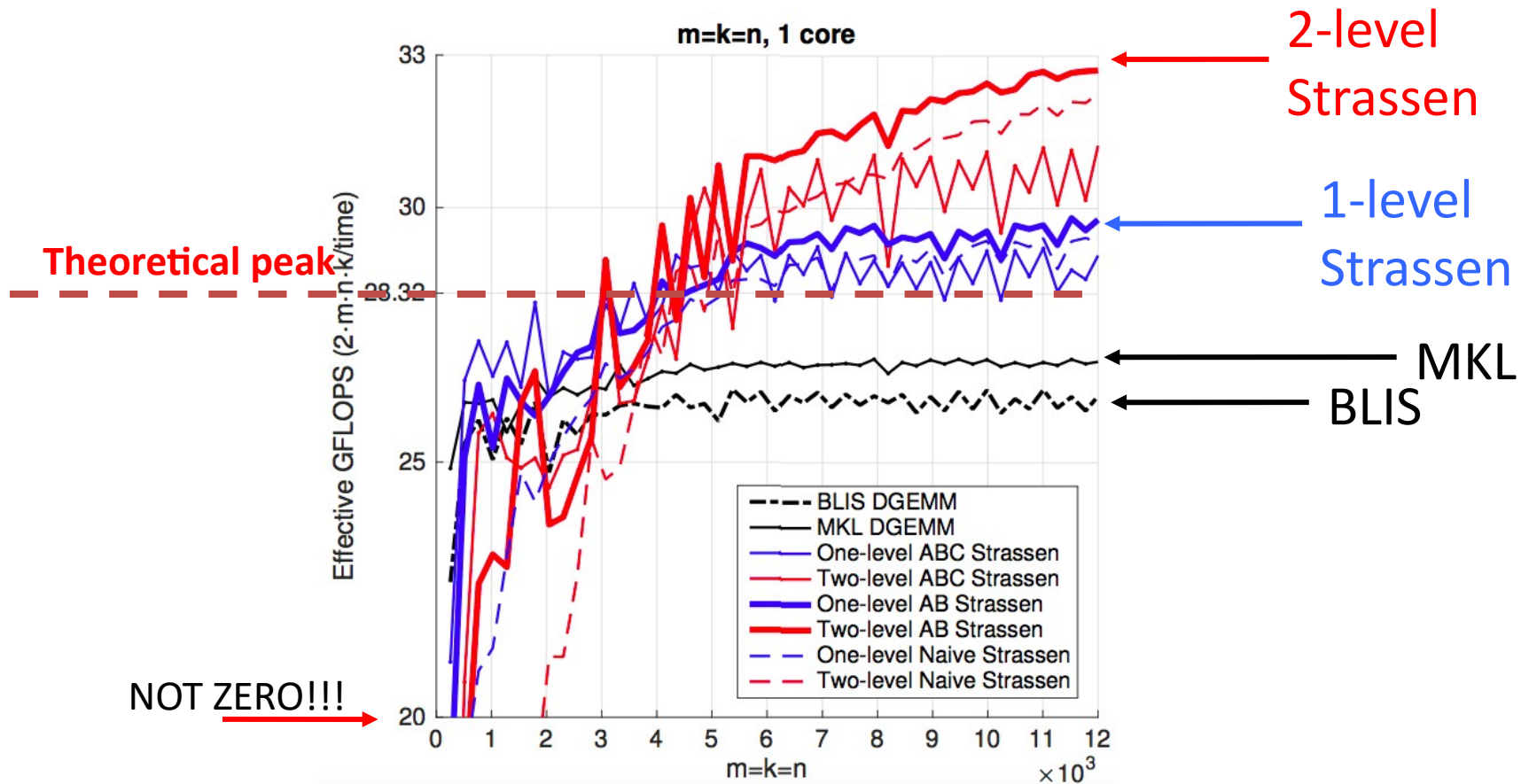
$$\begin{array}{ll} M_0 = (A_{00} + A_{11})(B_{00} + B_{11}); & C_{00} += \alpha M_0; C_{11} += \alpha M_0; \\ M_1 = (A_{10} + A_{11})B_{00}; & C_{10} += \alpha M_1; C_{11} -= \alpha M_1; \\ M_2 = A_{00}(B_{01} - B_{11}); & C_{01} += \alpha M_2; C_{11} += \alpha M_2; \\ M_3 = A_{11}(B_{10} - B_{00}); & C_{00} += \alpha M_3; C_{10} += \alpha M_3; \\ M_4 = (A_{00} + A_{01})B_{11}; & C_{01} += \alpha M_4; C_{00} -= \alpha M_4; \\ M_5 = (A_{10} - A_{00})(B_{00} + B_{01}); & C_{11} += \alpha M_5; \\ M_6 = (A_{01} - A_{11})(B_{10} + B_{11}); & C_{00} += \alpha M_6; \end{array}$$

$$M = \alpha(X + \delta Y)(V + \epsilon W); \quad C += \gamma_0 M; \quad D += \gamma_1 M;$$

$$M = \alpha(X + \delta Y)(V + \epsilon W); \quad C += \gamma_0 M; \quad D += \gamma_1 M;$$



Performance of Strassen's algorithm



J. Huang, T.M. Smith, G.M. Henry, R.A. van de Geijn. Strassen's Algorithm Reloaded. SC'16. 2016

Debunking conventional wisdom

Conventional wisdom

- Only works for large matrices
- Matrices must be squarish
- Requires extra workspace
- Doesn't parallelize well
- Requires new interface
- Won't work on GPUs except for very large matrices

Strassen Reloaded

- Already works for smallish matrices
- Works already for rank-k update
- No extra workspace
- Parallelizes just fine
- Doesn't require new interface
- Works on GPUs for smallish matrices

Strassen's algorithm

.

J. Huang, T.M. Smith, G.M. Henry, R.A. van de Geijn. Strassen's Algorithm Reloaded. SC'16. 2016.

J. Huang, L. Rice, D.A. Matthews, R.A. van de Geijn. Generating Families of Practical Fast Matrix Multiplication Algorithms. IPDPS'17. 2017

J. Huang, D.A. Matthews, R.A. van de Geijn. Strassen's algorithm for tensor contraction. SISC. 2018.

J. Huang, C. Yu, R.A. van de Geijn. Strassen's algorithm reloaded for GPUs. ACM TOMS. 2020